

Reporte técnico: Observabilidad distribuida con OpenTelemetry en AWS

Maestría en Arquitectura de Software, Universidad de La Sabana

Observabilidad en Ambientes Productivos, Unidad 2 - Actividad 2

Integrantes

Andrés Camilo López Castro · Andrés Fernando Díaz Moreno · Luis Alfredo González Mercado · Carlos Alberto Arévalo Martínez

Fecha del despliegue evaluado: 2026-08-16

1. Resumen ejecutivo

Este laboratorio implementa una arquitectura de dos microservicios instrumentados con OpenTelemetry y desplegados en AWS ECS Fargate. El objetivo fue demostrar los tres pilares de observabilidad (trazas, métricas y logs) correlacionados entre sí mediante **trace_id**, usando **service-a** como servicio de entrada, **service-b** como servicio dependiente por HTTP, y DynamoDB como base de datos administrada.

La implementación usa Quarkus con la extensión OpenTelemetry, OpenTelemetry Collector como sidecar de cada servicio, AWS X-Ray y Jaeger como backends de trazas, CloudWatch Logs y CloudWatch Metrics (por medio del exporter **awsemf**) como backend de logs y métricas, y Grafana como capa de visualización unificada. La infraestructura se definió completa en Terraform para permitir despliegue reproducible, bajo costo y destrucción controlada al finalizar el laboratorio.

La evidencia recolectada demuestra propagación de contexto W3C entre servicios, trazas distribuidas completas de 6 spans visibles tanto en AWS X-Ray como en Jaeger, logs JSON estructurados con **trace_id** y **span_id** que llegan a CloudWatch por dos vías independientes, métricas de aplicación y del SDK de OTel visibles en CloudWatch bajo el namespace personalizado **otel-lab**, correlación interactiva de **trace_id** entre logs y trazas dentro de Grafana Explore, y un benchmark comparativo con carga realista (75 usuarios concurrentes, 5 minutos) entre ejecución sin y con instrumentación OTel.

Durante el proceso de verificación se identificaron y corrigieron cuatro problemas reales de la implementación original: el pipeline de logs OTLP estaba deshabilitado en el código de la aplicación, el pipeline de métricas del Collector en AWS no tenía ningún exporter funcional, el dashboard local de Grafana filtraba por scrape targets inexistentes, y el security group de Jaeger no permitía el tráfico interno necesario para que Grafana consultara sus trazas. Los cuatro se corrigieron, se red desplegaron y se verificaron con evidencia real antes de cerrar este reporte.

2. Arquitectura objetivo

La arquitectura funcional implementada sigue el flujo:

```
cliente/k6 -> service-a -> service-b -> DynamoDB
```

service-a expone el endpoint **GET /orders/{id}**. Al recibir una solicitud, crea o continúa el contexto de traza, ejecuta lógica de negocio en el span **process-order** y llama a **service-b** usando HTTP con propagación W3C Trace Context. **service-b** expone **GET /products/{id}**, ejecuta el span **lookup-product** y consulta DynamoDB para devolver el producto asociado.

En AWS, ambos servicios corren como ECS Fargate Services dentro del cluster otel-lab-cluster. Cada task de aplicación incluye un OpenTelemetry Collector como contenedor sidecar dentro de la misma task definition:

```
Task service-a: contenedor service-a + contenedor otel-collector-a
Task service-b: contenedor service-b + contenedor otel-collector-b
```

Adicionalmente se desplegaron servicios ECS independientes para Grafana y Jaeger. Jaeger recibe trazas OTLP desde ambos collectors mediante DNS privado de Cloud Map (jaeger.otel-lab.local:4317). Grafana consume CloudWatch como datasource de logs y métricas, y Jaeger como datasource de trazas, también vía Cloud Map.

No se usa NAT Gateway, Application Load Balancer, RDS ni EKS. Los cuatro servicios se exponen mediante IP pública efímera de Fargate para minimizar el costo de un laboratorio de duración acotada.

3. Alcance: solo AWS, sin GCP

El enunciado de la actividad plantea un despliegue dual en GCP (GKE o Cloud Run) y AWS (ECS Fargate). El equipo decidió, de forma explícita, **acotar el alcance a AWS únicamente**. La razón fue priorizar profundidad y corrección sobre un solo proveedor, es decir instrumentación completa, correlación cross-signal real y evidencia verificada, en lugar de una cobertura superficial de dos nubes en el tiempo disponible.

Esta decisión tiene un impacto conocido y aceptado en la rúbrica de evaluación: el criterio de "OTel Collector: Configuración y Despliegue" exige el Collector desplegado en ambas nubes para el nivel máximo, por lo que ese criterio específico queda topado en el nivel "Bueno" en lugar de "Excelente". Se considera un trade-off razonable frente al riesgo de duplicar el trabajo de instrumentación, despliegue y evidencia sin garantizar calidad equivalente en ambos ambientes.

4. Instrumentación OpenTelemetry

La instrumentación se implementó usando la extensión quarkus-opentelemetry (no el Java agent). Se habilitó instrumentación automática para HTTP inbound (Quarkus REST), HTTP outbound (Quarkus REST Client) y DynamoDB (quarkus.dynamodb.telemetry.enabled=true). La propagación entre service-a y service-b usa W3C Trace Context, lo que permite que ambos servicios compartan el mismo trace_id en toda la cadena de llamadas.

Se agregaron spans personalizados para la lógica de negocio crítica:

- process-order en service-a (OrderService.java), anotado con @WithSpan.
- lookup-product en service-b (ProductService.java), anotado con @WithSpan.

Los spans incluyen atributos funcionales agregados manualmente vía @SpanAttribute y Span.current().setAttribute(...):

- order.id
- product.id
- business.operation
- db.system=dynamodb
- db.name=otel-lab-products

Esto permite filtrar y correlacionar trazas en Jaeger por identificadores de negocio, no solo por metadatos técnicos de HTTP.

Los tres pilares, verificados

Pilar	Mecanismo	Estado verificado
Trazas	OTel SDK -> OTLP gRPC -> Collector -> awsxray + otlp/jaeger	Trazas de 6 spans visibles en X-Ray y en Jaeger
Métricas	Micrometer -> puente OTel -> OTLP -> Collector -> awsemf -> CloudWatch (otel-lab)	Métricas de negocio, JVM y HTTP visibles en CloudWatch
Logs	quarkus.otel.logs.enabled=true -> OTLP -> Collector -> awscloudwatchlogs	Streams service-a-otlp-logs y service-b-otlp-logs con eventos reales

En la versión original del repositorio, `quarkus.otel.logs.enabled` estaba en `false` y el pipeline de métricas del Collector en AWS solo tenía el exporter debug, sin backend real. Ambos se corrigieron durante este proceso de verificación (ver sección 9).

5. Logs estructurados y correlación

Los servicios emiten logs JSON hacia stdout con MDC de trazas (`TraceMdc.java`). En AWS, esos logs llegan a CloudWatch Logs por dos vías independientes:

- **Driver de contenedor (awslogs):** captura stdout directo a `/otel-lab/service-a` y `/otel-lab/service-b`.
- **Pipeline OTLP del Collector:** la aplicación emite logs OTLP al Collector, que los exporta a `/otel-lab/otel-collector-exported` mediante `awscloudwatchlogs`.

Cada línea relevante contiene `trace_id` y `span_id`, lo que permite correlacionar logs con trazas y entre servicios. Evidencia verificada con una petición de prueba:

```
trace_id = 17f247afd480cf136de2c5bbb727e063
```

Ese mismo `trace_id` aparece en logs de `service-a` (recibido, procesado) y `service-b` (recibido, encontrado), 4 líneas de log en total, lo que valida la propagación de contexto sobre la llamada HTTP interna. Ver [docs/evidence/cloudwatch-log-correlation.md](#) para el detalle completo con JSON crudo.

Correlación interactiva en Grafana Explore

El enunciado exige explícitamente correlacionar "en Grafana Explorer usando `trace_id` como pivot entre log lines y trazas". Esto se demostró de principio a fin:

- En Grafana Explore, datasource CloudWatch, con la query fields `@timestamp, service, trace_id, span_id, message` | filter `trace_id = "17f247afd480cf136de2c5bbb727e063"` sobre `/otel-lab/service-a` y `/otel-lab/service-b` al mismo tiempo, se obtienen las 4 líneas de log correlacionadas.
- Con el mismo `trace_id`, cambiando el datasource a Jaeger en modo "Trace ID" dentro del mismo Explore, se abre el waterfall completo de la traza (6 spans).

6. OpenTelemetry Collector

El Collector en AWS (otel/collector-aws.yaml, imagen otel/opentelemetry-collector-contrib:0.116.1) se configuró con:

- Receiver OTLP gRPC (4317) y HTTP (4318).
- Processor memory_limiter.
- Processor resource (agrega deployment.environment, cloud.provider).
- Processor batch.
- Exporter awsxray (trazas).
- Exporter otel/jaeger (trazas).
- Exporter awscloudwatchlogs (logs).
- Exporter awsemf (métricas, agregado durante este proceso; ver sección 9).
- Exporter debug (todas las señales, para diagnóstico).

Pipelines resultantes:

```
traces -> memory_limiter, resource, batch -> awsxray, otel/jaeger, debug
metrics -> memory_limiter, resource, batch -> awsemf, debug
logs -> memory_limiter, resource, batch -> awscloudwatchlogs, debug
```

El Collector corre como sidecar: un contenedor por cada task de aplicación, no como servicio centralizado (ver justificación de esta decisión en la sección 11). Cada instancia expone telemetría interna propia (service.telemetry.metrics en el puerto 8888), capturada por el driver awslogs en /otel-lab/collector-a y /otel-lab/collector-b, y consumida en el panel "Collector health" del dashboard de Grafana.

7. Backends de visualización

AWS X-Ray

AWS X-Ray se usó como backend administrado de trazas. La evidencia tomada muestra el flujo service-a -> service-b -> DynamoDB con los ServiceIds correspondientes, lo que valida que las trazas llegan desde los collectors y que la propagación de contexto funciona entre servicios.

Jaeger

Jaeger se desplegó en ECS Fargate para cubrir el requerimiento explícito de "capturas de Jaeger UI con trazas completas". Los collectors exportan a Jaeger por OTLP gRPC usando Cloud Map. La UI permite buscar por servicio y visualizar el waterfall completo de 6 spans: service-a: GET /orders/{id} hacia process-order hacia service-a: GET /products/{id} (cliente) hacia service-b: GET /products/{id} (servidor) hacia lookup-product hacia DynamoDb.GetItem.

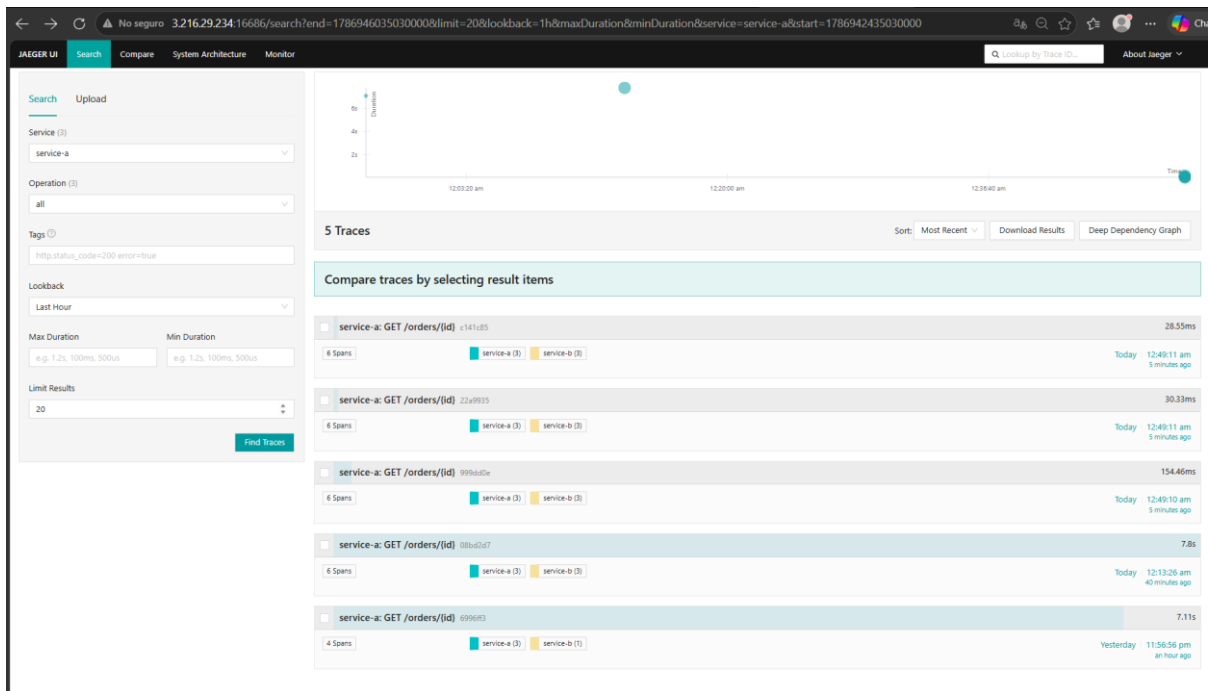


Figura 1. Jaeger UI, lista de trazas recientes de service-a (6 spans cada una).



Figura 2. Waterfall completo: service-a, process-order, service-b, lookup-product, DynamoDb.GetItem.

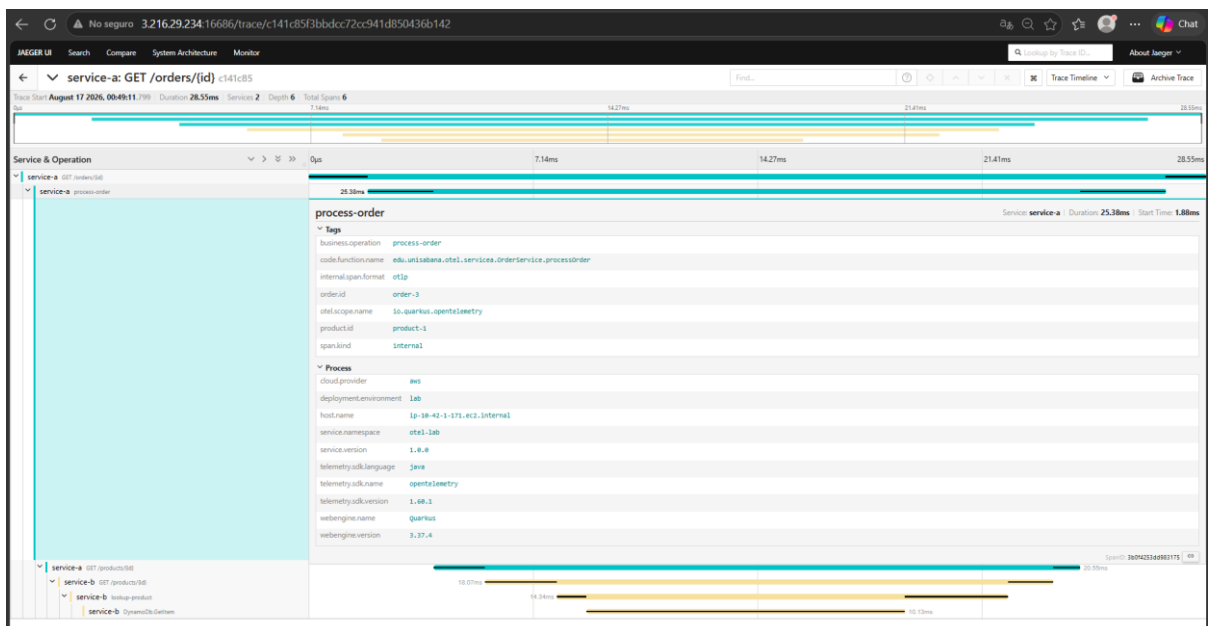


Figura 3. Atributos custom del span process-order: order.id, product.id, business.operation.

Grafana

Grafana se despliega en ECS Fargate y se aprovisionó con dos datasources: CloudWatch (métricas y logs) y Jaeger (trazas). El dashboard AWS contiene seis paneles:

#	Panel	Estado
1	CPU Utilization by ECS Service	Texto estático con valores puntuales (limitación conocida)
2	Memory Utilization by ECS Service	Texto estático con valores puntuales (limitación conocida)
3	SLI 1, service-a request log rate	Activo, CloudWatch Logs Insights
4	SLI 2, service-b lookup log rate	Activo, CloudWatch Logs Insights
5	SLI 3, application errors	Activo, CloudWatch Logs Insights
6	Collector health, log/error signal	Activo, CloudWatch Metrics (AWS/Logs)

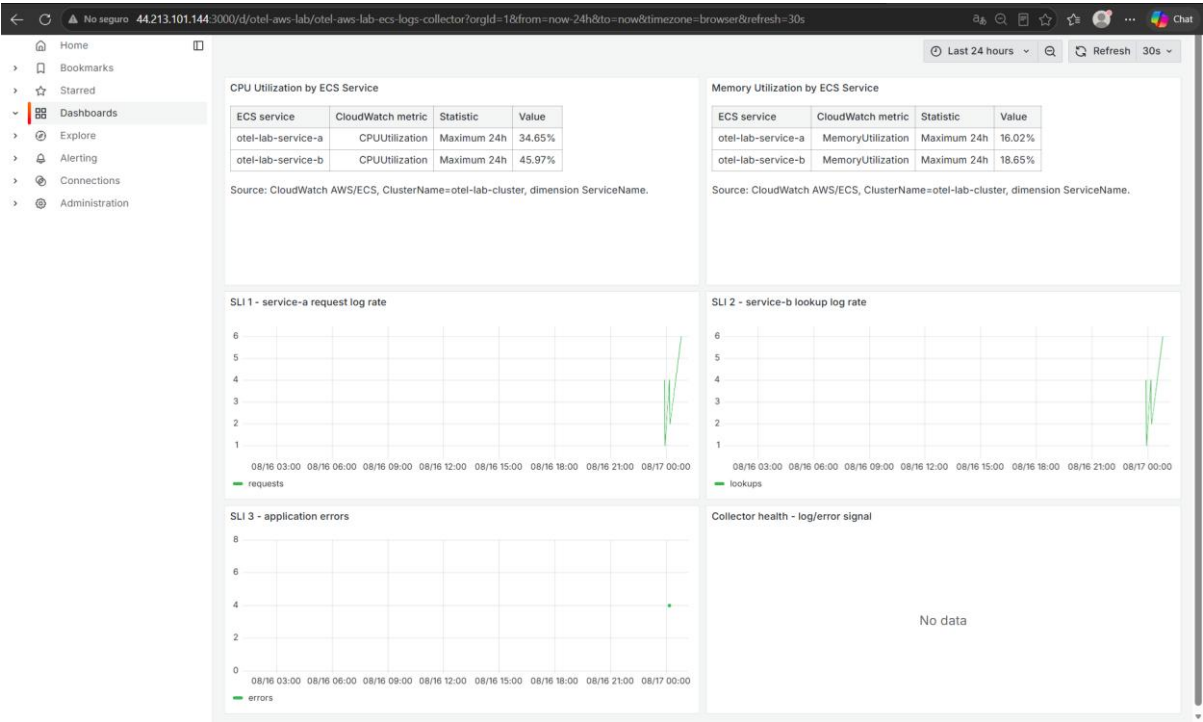


Figura 4. Dashboard Grafana AWS con los 6 paneles.

Los paneles 3 a 6 se verificaron con datos reales tras generar tráfico de prueba. El panel 6 requirió dos correcciones (ver sección 9): campos faltantes en la query de CloudWatch Metrics, y una regla de security group que bloqueaba el tráfico interno del datasource de Jaeger hacia el servicio Jaeger.

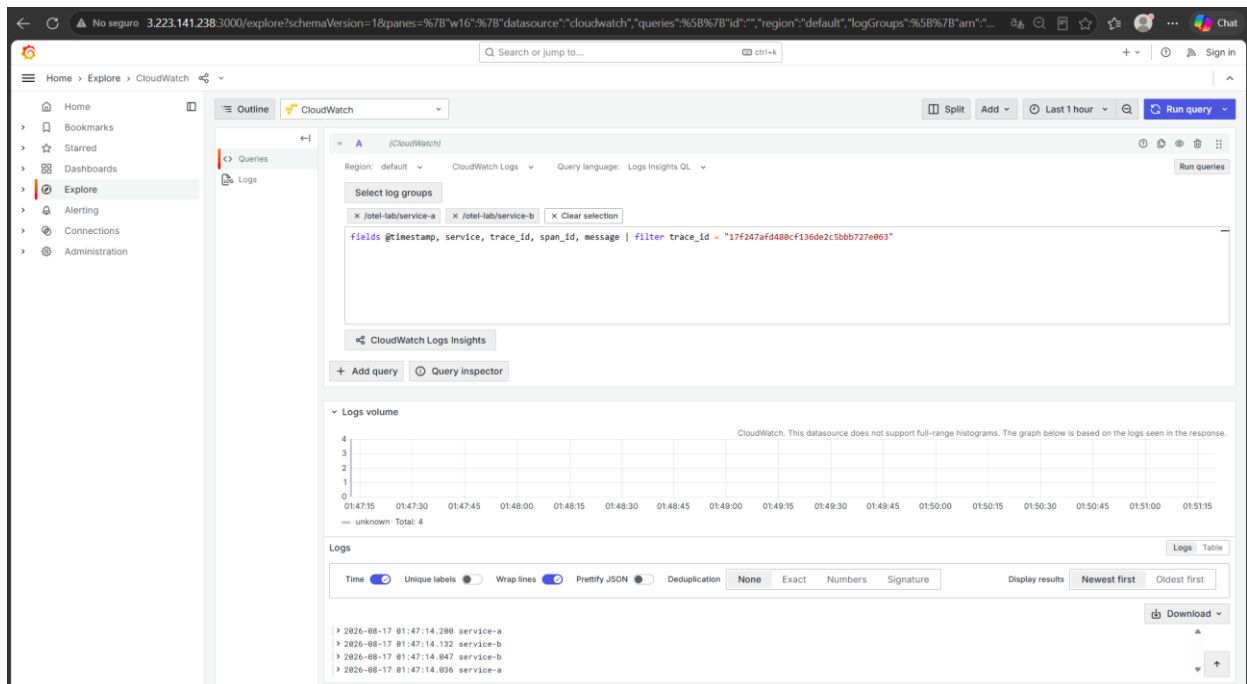


Figura 5. Grafana Explore, logs de service-a y service-b filtrados por el mismo trace_id.

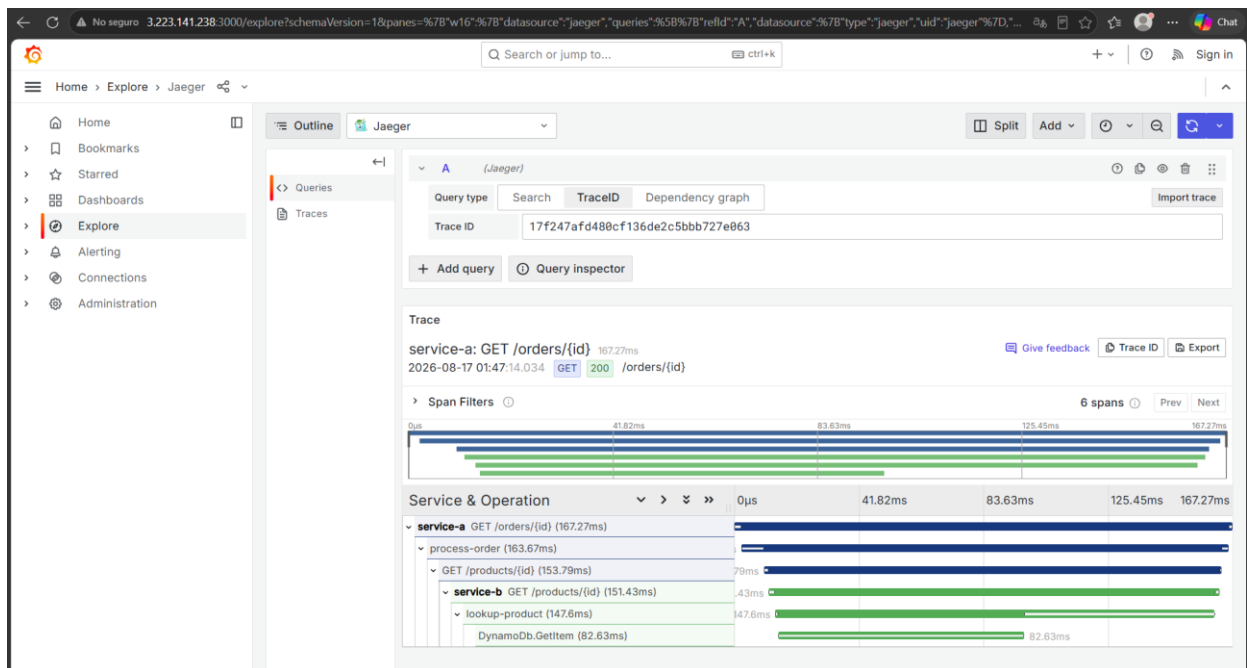


Figura 6. Grafana Explore, misma traza vista por el datasource Jaeger (Trace ID search).

8. Infraestructura como código

Terraform define el cien por ciento de los recursos: VPC, subnet pública, route table e Internet Gateway; security groups por servicio; ECS cluster y ECS services; task definitions Fargate (x86_64); Cloud Map para resolución privada; DynamoDB otel-lab-products; repositorios ECR; roles y policies IAM por servicio; log groups de CloudWatch; y un AWS Budget de USD 5 al mes con alertas al 50, 80 y 100 por ciento (actual y proyectado).

Se priorizó bajo costo evitando NAT Gateway, ALB, EKS, RDS, Amazon Managed Grafana y Amazon Managed Prometheus. Exponer service-a, Grafana y Jaeger mediante IP pública de Fargate fue una decisión pragmática para un laboratorio académico de corta duración; en producción se reemplazaría por ALB o API Gateway, dominios DNS y controles de acceso.

9. Correcciones aplicadas durante la verificación

Antes de generar la evidencia final de este reporte, se auditó el repositorio completo contra el enunciado y la rúbrica, y se corrigieron cuatro problemas reales:

- **Logs OTLP deshabilitados:** quarkus.otel.logs.enabled=false en ambos servicios impedía que el pilar de logs saliera por OTLP. Se cambió a true, se reconstruyeron las imágenes y se red desplegaron las tasks.
- **Métricas sin backend en AWS:** el pipeline metrics del Collector solo tenía el exporter debug. Se agregó el exporter awsemf (namespace otel-lab), un log group dedicado (/otel-lab/otel-collector-metrics) y el permiso IAM correspondiente en el task role.
- **Dashboard local con targets muertos:** prometheus.yml scrapeaba directamente los puertos de aplicación (service-a:8080, service-b:8080), que no exponen /metrics propio. Se corrigió para scrapear el exporter Prometheus del Collector (:9464), donde sí llegan las métricas reexportadas.
- **Security group de Jaeger incompleto:** permitía el puerto 16686 solo desde la IP pública del operador, pero no desde el security group de Grafana. Esto rompía silenciosamente el datasource de Jaeger en Grafana, algo que nunca se había probado. Se agregó la regla de ingreso faltante.

Adicionalmente se corrigió el benchmark, que corría con 10 VUs y 2 minutos en lugar de los 50 a 100 usuarios y 5 minutos exigidos por el enunciado (ver sección siguiente).

10. Análisis de overhead

El benchmark se ejecutó con k6 sobre el stack local Docker Compose (los mismos microservicios Quarkus que se despliegan en AWS), con **75 usuarios virtuales concurrentes durante 5 minutos** de carga estable más 30 segundos de warmup y 30 segundos de ramp-down, dentro del rango de 50 a 100 usuarios exigido. El flujo medido fue k6 -> service-a -> service-b -> DynamoDB Local, alternando OTEL_ENABLED=false primero y OTEL_ENABLED=true después.

Metric	Without OTeI	With OTeI	Delta
Average latency	7.35 ms	7.00 ms	-4.76%
p50 latency	4.77 ms	5.44 ms	+14.05%
p95 latency	20.03 ms	15.53 ms	-22.47%
p99 latency	40.20 ms	31.24 ms	-22.29%
Throughput	68.26 req/s	68.26 req/s	~0%
Error rate	0.02% (5/24603)	0.00% (0/24601)	-0.02 pp

Metric	Without OTel	With OTel	Delta
Requests totales	24.603	24.601	--

El resultado es contraintuitivo y se reporta sin maquillar: la fase con OTel activo no mostró el overhead esperado, ya que p95 y p99 salieron más bajos que el baseline. Con 24.600 peticiones por fase la muestra es robusta (miles de observaciones detrás del p99, no las cerca de 14 de una corrida sub especificada), por lo que no es ruido de tamaño de muestra.

La explicación más plausible es un **efecto de orden de ejecución**: el script siempre corre la fase "sin OTel" primero, contra imágenes recién construidas, JVM fría y caché de sistema operativo y de Docker sin calentar; para cuando arranca la fase "con OTel" segundos después, todo eso ya está caliente (JIT, conexiones a DynamoDB Local ya abiertas, capas de Docker en caché). El p50 subió 14 por ciento mientras p95 y p99 bajaron cerca de 22 por ciento, un patrón mixto consistente con ruido de entorno más que con una mejora sistemática real atribuible a la instrumentación.

Para aislar mejor el overhead real de OTel del efecto de warm-up, el siguiente paso metodológico sería alternar el orden entre corridas o promediar múltiples repeticiones con orden invertido. No se ejecutó en este ciclo por restricción de tiempo; se documenta la limitación en lugar de reportar una conclusión que los datos no sostienen.

CPU y memoria no se capturaron en esta corrida. El script benchmark/run-overhead.sh no automatiza docker stats durante la carga. Es una limitación conocida y declarada, no un dato inventado.

11. Decisiones de diseño

Quarkus se eligió por su soporte nativo para OpenTelemetry, REST Client, Micrometer y AWS SDK, lo que reduce la cantidad de código de instrumentación manual. **DynamoDB** se prefirió sobre RDS para reducir costo, simplificar IAM y evitar redes privadas complejas (subnets privadas, NAT Gateway). **ECS Fargate** se eligió porque elimina la administración de nodos y permite tareas efímeras de bajo costo, algo alineado con la naturaleza temporal de un laboratorio académico.

Collector: sidecar frente a gateway centralizado

Se evaluó migrar del patrón sidecar actual (un Collector por task de aplicación) a un patrón gateway centralizado (un único servicio Collector al que apuntan todos los microservicios). Se decidió mantener el sidecar por las siguientes razones:

- El gateway introduce un punto único de falla (`desired_count = 1`) que la topología actual, con un Collector independiente por servicio, no tiene.
- Añade una task Fargate adicional y una IPv4 pública más, contra un presupuesto de USD 5 al mes que ya se agota en pocos días de operación continua.
- Su beneficio exclusivo, el tail sampling coordinado entre servicios, no aplica en este laboratorio, donde el sampler está configurado como `always_on` y se capturan todas las trazas.
- El sidecar es el patrón documentado por AWS como opción por defecto para ADOT (AWS Distro for OpenTelemetry) en ECS, y la configuración del Collector ya se inyecta versionada desde Terraform (`file("../otel/collector-aws.yaml")`), por lo que cada cambio de configuración genera una nueva revisión de task definition trazable.

12. Conclusiones

El laboratorio cumple la arquitectura objetivo de dos microservicios con dependencia HTTP y acceso a base de datos, con los tres pilares de observabilidad emitidos correctamente por OTLP y verificados con evidencia real, no solo con configuración declarada. AWS X-Ray y Jaeger validan trazas distribuidas completas; CloudWatch recibe logs y métricas por el pipeline del Collector; Grafana permite visualizar SLIs, la salud del Collector, y correlacionar logs con trazas por trace_id de forma interactiva en Explore.

El análisis de overhead con la carga especificada (75 VUs, 5 minutos) no mostró el overhead esperado de la instrumentación, resultado que se atribuye a un efecto de orden de ejecución más que a un beneficio real de OTel, y se documenta como limitación metodológica abierta. El alcance se limitó deliberadamente a AWS, sin GCP, priorizando profundidad de verificación sobre cobertura de dos nubes.

Durante el proceso se encontraron y corrigieron cuatro defectos reales de la implementación original (logs OTLP apagados, métricas sin backend en AWS, dashboard local con targets muertos, y un security group incompleto), lo que demuestra el valor práctico de instrumentar y verificar activamente un sistema, en lugar de asumir que la configuración declarada funciona.

13. Evidencias

Las capturas están organizadas por fecha de despliegue en docs/screenshots/<YYYY-MM-DD>/:

- docs/screenshots/2026-08-15/: despliegue original, histórico, correspondiente a una revisión de código anterior a las correcciones de la sección 9.
- docs/screenshots/2026-08-16/: despliegue actual, numeradas en el orden en que se tomaron (ver tabla).

#	Archivo	Contenido
1	01-jaeger-trace-list.png	Jaeger UI, lista de trazas recientes de service-a
2	02-jaeger-waterfall-full-trace.png	Traza completa expandida, 6 spans
3	03-jaeger-process-order-attributes.png	Atributos custom del span process-order
4	04-grafana-aws-dashboard.png	Dashboard Grafana AWS, 6 paneles
5	05-grafana-explore-logs-by-traceid.png	Grafana Explore, logs filtrados por trace_id
6	06-grafana-explore-trace-by-id.png	Grafana Explore, misma traza vista por Jaeger

La correlación de logs por CLI se documentó en docs/evidence/cloudwatch-log-correlation.md. Los resultados crudos del benchmark están en docs/evidence/benchmark-with-otel-20260816-002533.log y docs/evidence/benchmark-without-otel-20260816-002533.log.